

Quy hoạch động cơ bản (Basic DP)

Nội dung

Chào mừng các bạn đến với Quy hoạch động (Dynamic Programming, viết tắt là DP) — một trong những kỹ thuật quan trọng nhất và mạnh mẽ nhất trong lập trình thi đấu! Đừng lo nếu bạn thấy DP khó hiểu lúc đầu — ai cũng vậy cả. Hãy đọc chậm, làm theo từng bước, và bạn sẽ nắm được.

1. Tại sao cần DP? Bắt đầu từ bài toán Fibonacci
 - Cách 1: Đệ quy thuần (CHẬM!)
2. Hai cách tiếp cận DP
 - Cách A: Ghi nhớ (Memoization) — Tiếp cận top-down
 - Cách B: Bảng phương (Tabulation) — Tiếp cận bottom-up
 - So sánh hai cách
3. Ba bước giải bài DP
4. Ví dụ chi tiết: Bài toán đồng xu (Coin Change)
 - Bước 1: Xác định trạng thái
 - Bước 2: Viết công thức truy hồi
 - Bước 3: Cơ sở
 - Truy vết từng bước
 - Code hoàn chỉnh
5. DP trên lưới (Grid Paths)
 - Bước 1: Trạng thái
 - Bước 2: Công thức truy hồi
 - Bước 3: Cơ sở
 - Truy vết trực quan
 - Code hoàn chỉnh
6. Dãy con tăng dài nhất (LIS — Longest Increasing Subsequence)
 - Bước 1: Trạng thái
 - Bước 2: Công thức truy hồi
 - Bước 3: Cơ sở
 - Truy vết từng bước
 - Code hoàn chỉnh
7. Bài toán ba lô (0/1 Knapsack)
 - Tại sao phải có chiều i ?

- Một cách nghĩ rất quan trọng
- Tóm tắt trực giác
- Bước 1: Trạng thái (phiên bản 2D)
- Bước 2: Công thức truy hồi
- Bước 3: Cơ sở
- Truy vết bảng DP
- Code (2D DP)
- Tối ưu bộ nhớ: Bước trung gian — `dp_old` / `dp_new`
- Tối ưu bộ nhớ: Gộp thành mảng 1D

8. Khi nào dùng DP?

9. Sai lầm phổ biến khi làm DP

10. Mẹo luyện tập

1. Tại sao cần DP? Bắt đầu từ bài toán Fibonacci

Trước khi học DP, hãy xem một ví dụ quen thuộc: dãy Fibonacci.

$$F(0) = 0, \quad F(1) = 1, \quad F(n) = F(n-1) + F(n-2)$$

Cách 1: Đệ quy thuần (CHẬM!)

C++

```
int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}
```

Python

```
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)
```

Cách này rất chậm. Tại sao? Hãy xem cây đệ quy khi tính `fib(5)`:

```

      fib(5)
     /    \
  fib(4)  fib(3)
  /  \    /  \
fib(3) fib(2) fib(2) fib(1)
 /  \  /  \  /  \
fib(2) fib(1) ...
 /    \

```

```
fib(1) fib(0)
```

Nhìn thấy không? `fib(3)` được tính 2 lần, `fib(2)` được tính 3 lần, `fib(1)` được tính 5 lần! Với n lớn, số lần tính lặp lại tăng theo cấp số nhân. Độ phức tạp là $O(2^n)$ — với $n = 50$, cần khoảng 10^{15} phép tính, máy tính chạy cả ngày cũng không xong!

Vấn đề cốt lõi: Cùng một bài toán con (ví dụ `fib(3)`) bị tính đi tính lại nhiều lần. Đây gọi là **bài toán con chồng chéo (overlapping subproblems)**.

Giải pháp: Lần đầu tính xong → lưu kết quả lại → lần sau cần thì lấy ra dùng, không tính lại nữa. Đó chính là ý tưởng cốt lõi của DP!

2. Hai cách tiếp cận DP

Cách A: Ghi nhớ (Memoization) — Tiếp cận top-down

Vẫn dùng đệ quy, nhưng thêm một bảng ghi nhớ. Trước khi tính, kiểm tra xem đã tính trước đó chưa.

C++

```
#include <bits/stdc++.h>
using namespace std;

long long memo[105];
bool computed[105];

long long fib(int n) {
    if (n <= 1) return n;
    if (computed[n]) return memo[n];
    computed[n] = true;
    memo[n] = fib(n - 1) + fib(n - 2);
    return memo[n];
}

int main() {
    int n;
    cin >> n;
    memset(computed, false, sizeof computed);
    cout << fib(n) << endl;
}
```

Python

```
memo = {}

def fib(n):
    if n <= 1:
        return n
    if n in memo:
        return memo[n]
    memo[n] = fib(n - 1) + fib(n - 2)
```

```

    return memo[n]

n = int(input())
print(fib(n))

```

Cây đệ quy bây giờ (tính `fib(5)`):

```

          fib(5)
        /      \
      fib(4)    fib(3) <-- đã lưu, trả luôn!
    /      \
  fib(3)    fib(2)      <-- đã lưu, trả luôn!
 /      \
fib(2)    fib(1)
 /      \
fib(1)  fib(0)

```

Mỗi giá trị chỉ tính đúng 1 lần $\rightarrow$ Độ phức tạp $O(n)$. Nhanh hơn hàng tỷ lần!

Cách B: Bảng phương (Tabulation) — Tiếp cận bottom-up

Thay vì đệ quy, ta dùng vòng lặp tính từ nhỏ đến lớn:

C++

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<long long> dp(n + 1);
    dp[0] = 0;
    dp[1] = 1;
    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }
    cout << dp[n] << endl;
}

```

Python

```

n = int(input())
if n == 0:
    print(0)
else:
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]

```

```
print(dp[n])
```

Quá trình tính:

$$\begin{aligned}dp[0] &= 0 \\dp[1] &= 1 \\dp[2] &= dp[1] + dp[0] = 1 + 0 = 1 \\dp[3] &= dp[2] + dp[1] = 1 + 1 = 2 \\dp[4] &= dp[3] + dp[2] = 2 + 1 = 3 \\dp[5] &= dp[4] + dp[3] = 3 + 2 = 5\end{aligned}$$

Đơn giản, nhanh, dễ hiểu!

So sánh hai cách

	TOP-DOWN (MEMOIZATION)	BOTTOM-UP (TABULATION)
Cách viết	Đệ quy + ghi nhớ	Vòng lặp
Ưu điểm	Dễ nghĩ, chỉ tính trạng thái cần	Thường nhanh hơn (không có overhead đệ quy)
Nhược điểm	Có thể bị tràn stack	Phải xác định đúng thứ tự tính
Khi nào dùng	Khi không chắc thứ tự tính	Hầu hết các trường hợp (khuyến khích)

Trong lập trình thi đấu, cách bottom-up phổ biến hơn vì nhanh hơn và tránh được stack overflow. Nhưng top-down đôi khi dễ nghĩ hơn cho bài phức tạp. Bạn nên biết cả hai!

3. Ba bước giải bài DP

Mọi bài DP đều tuân theo 3 bước:

BƯỚC	CÂU HỎI CẦN TRẢ LỜI	VÍ DỤ (FIBONACCI)
1. Xác định trạng thái	$dp[i]$ (hoặc $dp[i][j]...$) đại diện cho cái gì?	$dp[i]$ = số Fibonacci thứ i
2. Viết công thức truy hồi	$dp[i]$ được tính từ những trạng thái nào?	$dp[i] = dp[i-1] + dp[i-2]$
3. Xác định cơ sở	Những giá trị ban đầu nào ta biết sẵn?	$dp[0] = 0, dp[1] = 1$

Mẹo: Bước 1 là quan trọng nhất và khó nhất. Nếu xác định sai trạng thái, mọi thứ sau đó đều sai. Hãy dành nhiều thời gian cho bước này!

4. Ví dụ chi tiết: Bài toán đồng xu (Coin Change)

Đề bài: Cho k loại đồng xu có mệnh giá $c_1, c_2, \dots, c_k$ (mỗi loại có vô hạn). Tìm số đồng xu ít nhất cần dùng để tạo tổng đúng bằng n . Nếu không thể, in -1 .

Ví dụ: Đồng xu mệnh giá $\{1, 3, 4\}$, $n = 6$.

- Dùng đồng 1: $1 + 1 + 1 + 1 + 1 + 1 = 6 \rightarrow 6$ đồng
- Dùng đồng 3: $3 + 3 = 6 \rightarrow 2$ đồng
- Dùng 1 đồng 4 + đồng 1 + ... Nhiều cách khác
- Đáp án tối ưu: $3 + 3 = 6 \rightarrow 2$ đồng

Tại sao Greedy sai? Nếu dùng tham lam (luôn chọn đồng lớn nhất trước): chọn 4, còn 2, chọn 1, còn 1, chọn 1 $\rightarrow 4 + 1 + 1 = 6 \rightarrow 3$ đồng. Nhưng đáp án tối ưu là 2 đồng ($3 + 3$). Greedy cho sai!

Bước 1: Xác định trạng thái

$dp[i]$ = số đồng xu ít nhất cần dùng để tạo tổng đúng bằng i .

- $dp[0]$ =? Để tạo tổng 0, không cần đồng nào $\rightarrow dp[0] = 0$.
- $dp[6]$ =? Đó chính là đáp án ta cần tìm.

Bước 2: Viết công thức truy hồi

Để tạo tổng i , đồng xu cuối cùng ta dùng phải là một trong $c_1, c_2, \dots, c_k$.

- Nếu đồng cuối cùng là c_j (với $c_j \leq i$), thì trước đó ta đã tạo tổng $i - c_j$, cần $dp[i - c_j]$ đồng. Thêm đồng c_j nữa $\rightarrow$ tổng cộng $dp[i - c_j] + 1$ đồng.
- Ta muốn ít đồng nhất $\rightarrow$ lấy min trên tất cả lựa chọn:

$$dp[i] = \min_{j=1}^k (dp[i - c_j] + 1) \quad \text{với mọi } c_j \leq i$$

Bước 3: Cơ sở

- $dp[0] = 0$ (tổng 0 cần 0 đồng)
- $dp[i] = +\infty$ ban đầu cho $i > 0$ (chưa biết có tạo được không)

Truy vết từng bước

Đồng xu: $\{1, 3, 4\}$, $n = 6$.

Ban đầu: $dp = [0, \text{INF}, \text{INF}, \text{INF}, \text{INF}, \text{INF}, \text{INF}]$

$i = 1$:

Thử $c=1$: $dp[1-1] + 1 = dp[0] + 1 = 0 + 1 = 1$ (nhận)

Thử $c=3$: $3 > 1 \rightarrow$ bỏ qua

Thử $c=4$: $4 > 1 \rightarrow$ bỏ qua

$dp[1] = 1$

$i = 2$:

Thử $c=1$: $dp[2-1] + 1 = dp[1] + 1 = 1 + 1 = 2$ (nhận)

Thử $c=3$: $3 > 2 \rightarrow$ bỏ qua

Thử $c=4$: $4 > 2 \rightarrow$ bỏ qua

$dp[2] = 2$

$i = 3$:

Thử $c=1$: $dp[3-1] + 1 = dp[2] + 1 = 2 + 1 = 3$

Thử $c=3$: $dp[3-3] + 1 = dp[0] + 1 = 0 + 1 = 1$ (tốt nhất!)

Thử $c=4$: $4 > 3 \rightarrow$ bỏ qua

$dp[3] = 1$

$i = 4$:

Thử $c=1$: $dp[4] + 1 = 1 + 1 = 2$

Thử $c=3$: $dp[1] + 1 = 1 + 1 = 2$

Thử $c=4$: $dp[0] + 1 = 0 + 1 = 1$ (tốt nhất!)

$dp[4] = 1$

$i = 5$:

Thử $c=1$: $dp[4] + 1 = 1 + 1 = 2$ (nhận)

Thử $c=3$: $dp[2] + 1 = 2 + 1 = 3$

Thử $c=4$: $dp[1] + 1 = 1 + 1 = 2$ (nhận)

$dp[5] = 2$

$i = 6$:

Thử $c=1$: $dp[5] + 1 = 2 + 1 = 3$

Thử $c=3$: $dp[3] + 1 = 1 + 1 = 2$ (tốt nhất!)

Thử $c=4$: $dp[2] + 1 = 2 + 1 = 3$

$dp[6] = 2$

Kết quả: $dp = [0, 1, 2, 1, 1, 2, 2]$. Đáp án là $dp[6] = 2$, tức dùng 2 đồng ($3 + 3$).

Code hoàn chỉnh

C++

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n, k;
    cin >> n >> k;
    vector<int> coins(k);
    for (int &c : coins) cin >> c;

    // dp[i] = số đồng xu ít nhất để tạo tổng i
    vector<int> dp(n + 1, 1e9); // 1e9 đại diện cho "không thể"
    dp[0] = 0; // Cơ sở: tổng 0 cần 0 đồng
```

```

    for (int i = 1; i <= n; i++) {
        for (int c : coins) {
            if (c <= i && dp[i - c] + 1 < dp[i]) {
                dp[i] = dp[i - c] + 1;
            }
        }
    }

    if (dp[n] >= 1e9) cout << -1 << endl;
    else cout << dp[n] << endl;
}

```

Python

```

n, k = map(int, input().split())
coins = list(map(int, input().split()))

INF = 10**9
dp = [INF] * (n + 1)
dp[0] = 0

for i in range(1, n + 1):
    for c in coins:
        if c <= i and dp[i - c] + 1 < dp[i]:
            dp[i] = dp[i - c] + 1

if dp[n] >= INF:
    print(-1)
else:
    print(dp[n])

```

Input mẫu:

```

6 3
1 3 4

```

Output mẫu:

```

2

```

5. DP trên lưới (Grid Paths)

Đề bài: Cho lưới $n \times n$. Một số ô là chướng ngại vật ('*'). Đếm số đường đi từ ô $(1, 1)$ (góc trên-trái) đến ô (n, n) (góc dưới-phải), chỉ được đi sang phải hoặc xuống dưới. In kết quả modulo $10^9 + 7$.

Ví dụ lưới 4×4 :

```

. . . .
. * . .
. . . *
. . . .

```


Bước 1: Trạng thái

$dp[i][j]$ = số đường đi từ $(1, 1)$ đến (i, j) .

Bước 2: Công thức truy hồi

Để đến ô (i, j) , ta chỉ có thể đến từ:

- Ô phía trên: $(i - 1, j)$ — đi xuống
- Ô bên trái: $(i, j - 1)$ — đi sang phải

Vậy:

$$dp[i][j] = dp[i - 1][j] + dp[i][j - 1]$$

Nếu (i, j) là chướng ngại vật: $dp[i][j] = 0$ (không thể đến đây).

Bước 3: Cơ sở

$dp[1][1] = 1$ (nếu $(1, 1)$ không phải chướng ngại vật).

Truy vết trực quan

Điền giá trị vào lưới (ô chướng ngại vật = 0):

Lưới: Bảng dp:

⋮ * ⋮ ⋮
⋮ ⋮ ⋮ *

1	1	1	1
1	0	1	2
1	1	2	0
1	2	4	4

Giải thích từng ô:

- $dp[1][1] = 1$ (cơ sở)
- Hàng đầu tiên: mỗi ô chỉ có thể đến từ bên trái $\rightarrow dp[1][j] = dp[1][j - 1] = 1$
- Cột đầu tiên: mỗi ô chỉ có thể đến từ trên $\rightarrow dp[i][1] = dp[i - 1][1] = 1$
- $dp[2][2] = 0$ vì $(2, 2)$ là chướng ngại vật
- $dp[2][3] = dp[1][3] + dp[2][2] = 1 + 0 = 1$
- $dp[2][4] = dp[1][4] + dp[2][3] = 1 + 1 = 2$
- $dp[3][3] = dp[2][3] + dp[3][2] = 1 + 1 = 2$
- $dp[3][4] = 0$ (chướng ngại vật)

- $dp[4][3] = dp[3][3] + dp[4][2] = 2 + 2 = 4$
- $dp[4][4] = dp[3][4] + dp[4][3] = 0 + 4 = 4$

Đáp án: 4 đường đi.

Code hoàn chỉnh

C++

```
#include <bits/stdc++.h>
using namespace std;

const int MOD = 1e9 + 7;

int main() {
    int n;
    cin >> n;
    vector<string> grid(n);
    for (auto &row : grid) cin >> row;

    vector<vector<long long>> dp(n, vector<long long>(n, 0));

    // Cơ sở
    if (grid[0][0] == '*') {
        cout << 0 << endl;
        return 0;
    }
    dp[0][0] = 1;

    // Điền bảng theo thứ tự từ trên xuống, trái qua phải
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (grid[i][j] == '*') {
                dp[i][j] = 0;
                continue;
            }
            if (i > 0) dp[i][j] = (dp[i][j] + dp[i - 1][j]) % MOD;
            if (j > 0) dp[i][j] = (dp[i][j] + dp[i][j - 1]) % MOD;
        }
    }

    cout << dp[n - 1][n - 1] << endl;
}
```

Python

```
MOD = 10**9 + 7
n = int(input())
grid = [input().strip() for _ in range(n)]
```

```

dp = [[0] * n for _ in range(n)]

if grid[0][0] == '*':
    print(0)
else:
    dp[0][0] = 1
    for i in range(n):
        for j in range(n):
            if grid[i][j] == '*':
                dp[i][j] = 0
                continue
            if i > 0:
                dp[i][j] = (dp[i][j] + dp[i - 1][j]) % MOD
            if j > 0:
                dp[i][j] = (dp[i][j] + dp[i][j - 1]) % MOD
    print(dp[n - 1][n - 1])

```

Lưu ý: Nhớ lấy modulo $10^9 + 7$ ở mỗi bước cộng để tránh tràn số!

6. Dãy con tăng dài nhất (LIS — Longest Increasing Subsequence)

Đề bài: Cho dãy $a_1, a_2, \dots, a_n$. Tìm dãy con (không cần liên tiếp) tăng nghiêm ngặt dài nhất.

Ví dụ: $a = [3, 1, 4, 1, 5, 9, 2, 6]$. Dãy con tăng dài nhất là $[1, 4, 5, 9]$ hoặc $[1, 4, 5, 6]$ hoặc $[3, 4, 5, 9]$ (LIS = 4).

Lưu ý: "Dãy con" (subsequence) khác "đoạn con" (subarray). Dãy con không cần liên tiếp, chỉ cần giữ nguyên thứ tự.

Bước 1: Trạng thái

$dp[i]$ = độ dài dãy con tăng dài nhất kết thúc tại vị trí i .

Tại sao lại "kết thúc tại i "? Vì khi mở rộng dãy con, ta cần biết phần tử cuối cùng là gì để so sánh.

Bước 2: Công thức truy hồi

Để mở rộng dãy con kết thúc tại i , ta tìm tất cả vị trí $j < i$ sao cho $a[j] < a[i]$ (để nối tiếp được):

$$dp[i] = \max(dp[j] + 1) \quad \text{với mọi } j < i \text{ sao cho } a[j] < a[i]$$

Nếu không có j nào thỏa mãn, $dp[i] = 1$ (dãy con chỉ gồm $a[i]$).

Bước 3: Cơ sở

$dp[i] = 1$ cho mọi i (mỗi phần tử tự nó là dãy con tăng độ dài 1).

Truy vết từng bước

$a = [3, 1, 4, 1, 5, 9, 2, 6]$ (đánh số từ 0).

$i=0, a[0]=3$: Không có $j < 0 \rightarrow dp[0] = 1$ [3]
 $i=1, a[1]=1$: $j=0$: $a[0]=3 > 1$ (loại) $\rightarrow dp[1] = 1$ [1]
 $i=2, a[2]=4$: $j=0$: $a[0]=3 < 4$ (thỏa) $dp[0]+1=2$
 $j=1$: $a[1]=1 < 4$ (thỏa) $dp[1]+1=2$
 $\rightarrow dp[2] = 2$ [3,4] hoặc [1,4]
 $i=3, a[3]=1$: $j=0$: $a[0]=3 > 1$ (loại)
 $j=1$: $a[1]=1 = 1$ (loại, cần nghiêm ngặt <)
 $j=2$: $a[2]=4 > 1$ (loại)
 $\rightarrow dp[3] = 1$ [1]
 $i=4, a[4]=5$: $j=0$: $3 < 5$ (thỏa) $dp[0]+1=2$
 $j=1$: $1 < 5$ (thỏa) $dp[1]+1=2$
 $j=2$: $4 < 5$ (thỏa) $dp[2]+1=3$ <-- tốt nhất!
 $j=3$: $1 < 5$ (thỏa) $dp[3]+1=2$
 $\rightarrow dp[4] = 3$ [1,4,5] hoặc [3,4,5]
 $i=5, a[5]=9$: $dp[0]+1=2, dp[1]+1=2, dp[2]+1=3,$
 $dp[3]+1=2, dp[4]+1=4$ <-- tốt nhất!
 $\rightarrow dp[5] = 4$ [1,4,5,9]
 $i=6, a[6]=2$: $j=1$: $1 < 2$ (thỏa) $dp[1]+1=2$
 $j=3$: $1 < 2$ (thỏa) $dp[3]+1=2$
 $\rightarrow dp[6] = 2$ [1,2]
 $i=7, a[7]=6$: $dp[0]+1=2, dp[1]+1=2, dp[2]+1=3,$
 $dp[3]+1=2, dp[4]+1=4, dp[6]+1=3$
 $\rightarrow dp[7] = 4$ [1,4,5,6]

Bảng dp :

a :	3	1	4	1	5	9	2	6
dp :	1	1	2	1	3	4	2	4

$LIS = \max(dp) = 4$.

Code hoàn chỉnh

C++

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> a(n);
    for (int &x : a) cin >> x;

    vector<int> dp(n, 1); // Cơ sở: mỗi phần tử tự nó là dãy con độ dài 1

    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
```

```

        if (a[j] < a[i]) {
            dp[i] = max(dp[i], dp[j] + 1);
        }
    }

    int lis = *max_element(dp.begin(), dp.end());
    cout << lis << endl;
}

```

Python

```

n = int(input())
a = list(map(int, input().split()))

dp = [1] * n

for i in range(1, n):
    for j in range(i):
        if a[j] < a[i]:
            dp[i] = max(dp[i], dp[j] + 1)

print(max(dp))

```

Độ phức tạp: $O(n^2)$. Với $n \leq 5000$, cách này đủ nhanh. (Có cách $O(n \log n)$ nhưng thuộc mức nâng cao hơn.)

7. Bài toán ba lô (0/1 Knapsack)

Đề bài: Có n đồ vật. Đồ vật thứ i có trọng lượng w_i và giá trị v_i . Ba lô chứa được tối đa W kg. Mỗi đồ vật chỉ lấy hoặc không (không lấy một phần). Tìm tổng giá trị lớn nhất có thể mang đi.

Ví dụ: $W = 7$, $n = 4$ đồ vật:

ĐỒ VẬT	TRỌNG LƯỢNG (w_i)	GIÁ TRỊ (v_i)
1	1	1
2	3	4
3	4	5
4	5	7

Knapsack là bài rất điển hình cho tư duy "ra quyết định từng bước". Khi đứng trước một đồ vật, ta chỉ có 2 lựa chọn:

- Lấy nó
- Không lấy nó

Điều này gợi ý rằng ta nên xử lý các đồ vật theo thứ tự, và tại mỗi bước chỉ cần biết:

1. Ta đã xét đến bao nhiêu đồ vật.
2. Ba lô còn (hoặc có) bao nhiêu sức chứa.

Vì vậy, trạng thái tự nhiên nhất là:

$dp[i][j]$ = giá trị lớn nhất khi xét i đồ vật đầu tiên, với sức chứa tối đa j .

Tại sao trạng thái này hợp lý?

- Nếu biết ta đang ở đồ vật thứ i .
- Và biết ba lô có sức chứa tối đa j .
- Thì mọi quyết định tiếp theo chỉ phụ thuộc vào 2 thông tin đó.

Ta không cần nhớ chính xác đã lấy những đồ vật nào, chỉ cần nhớ: đã xét đến đâu và còn bao nhiêu khả năng chứa. Đó là dấu hiệu của một trạng thái DP tốt: nó chứa đủ thông tin để ra quyết định tiếp, nhưng không chứa thông tin thừa.

Tại sao phải có chiều i ?

Nhiều bạn mới học hay hỏi: "*Sao không đặt luôn $dp[j]$ là đáp án với sức chứa j ?*"

Thật ra có thể, nhưng đó là phiên bản tối ưu bộ nhớ sau này. Ban đầu, dùng $dp[i][j]$ dễ hiểu hơn vì:

- i cho biết ta đang xét đồ vật nào.
- Khi đó, với đồ vật thứ i , ta có đúng 2 lựa chọn:
 - Không lấy: quay về bài toán với $i - 1$ đồ vật.
 - Lấy: cũng quay về bài toán với $i - 1$ đồ vật, nhưng giảm sức chứa đi w_i .

Nói cách khác, chiều i giúp ta diễn đạt rất rõ ý tưởng: "*Đáp án hiện tại được xây từ đáp án của bài toán nhỏ hơn.*"

Một cách nghĩ rất quan trọng

Hãy tưởng tượng:

- $dp[i][j]$ là đáp án tốt nhất sau khi đã quyết định xong với i đồ vật đầu tiên.
- Đồ vật thứ i là đồ vật "mới nhất" vừa được xét.
- Không lấy đồ vật $i \rightarrow$ đáp án giữ nguyên như cũ: $dp[i - 1][j]$.
- Lấy đồ vật $i \rightarrow$ ta phải dành ra w_i đơn vị sức chứa $\rightarrow$ phần còn lại là bài toán: dùng $i - 1$ đồ vật đầu để đạt giá trị tốt nhất với sức chứa $j - w_i \rightarrow$ giá trị nhận được là $dp[i - 1][j - w_i] + v_i$.

Từ trực giác này, công thức truy hồi gần như "tự rơi ra":

$$dp[i][j] = \max(dp[i - 1][j], dp[i - 1][j - w_i] + v_i)$$

Tóm tắt trực giác

Trạng thái Knapsack có dạng $dp[i][j]$ vì bài toán có đúng 2 chiều thông tin quan trọng:

- Đã xét bao nhiêu đồ vật.
- Còn/có bao nhiêu sức chứa.

Đây là cách mô tả ngắn gọn nhưng đủ mạnh để biểu diễn mọi lời giải con của bài toán.

Bước 1: Trạng thái (phiên bản 2D)

$dp[i][j]$ = giá trị lớn nhất có thể đạt được khi xét i đồ vật đầu tiên và ba lô có sức chứa tối đa j .

Bước 2: Công thức truy hồi

Với đồ vật thứ i , ta có 2 lựa chọn:

- Không lấy đồ vật i : $dp[i][j] = dp[i-1][j]$.
- Lấy đồ vật i (nếu $j \geq w_i$): $dp[i][j] = dp[i-1][j-w_i] + v_i$.

Lấy max:

$$dp[i][j] = \max(dp[i-1][j], dp[i-1][j-w_i] + v_i)$$

Bước 3: Cơ sở

$dp[0][j] = 0$ cho mọi j (chưa xét đồ vật nào $\rightarrow$ giá trị = 0).

Truy vết bảng DP

Bảng $dp[i][j]$ (hàng = đồ vật, cột = sức chứa 0-7):

j :	0	1	2	3	4	5	6	7
$i = 0$ (-)	0	0	0	0	0	0	0	0
$i = 1$ ($w = 1, v = 1$)	0	1	1	1	1	1	1	1
$i = 2$ ($w = 3, v = 4$)	0	1	1	4	5	5	5	5
$i = 3$ ($w = 4, v = 5$)	0	1	1	4	5	6	6	9
$i = 4$ ($w = 5, v = 7$)	0	1	1	4	5	7	8	9

Giải thích một số ô:

- $dp[2][3]$: Xét vật 1, 2; sức chứa 3.
 - Lấy vật 2 ($w = 3, v = 4$): $dp[1][0] + 4 = 0 + 4 = 4$.
 - Không lấy: $dp[1][3] = 1$.
 - Max = 4.
- $dp[2][4]$:
 - Lấy vật 2: $dp[1][1] + 4 = 1 + 4 = 5$.

- Không lấy: $dp[1][4] = 1$.
- Max = 5.
- $dp[3][7]$:
 - Lấy vật 3 ($w = 4$): $dp[2][3] + 5 = 4 + 5 = 9$.
 - Không lấy: $dp[2][7] = 5$.
 - Max = 9.

Đáp án: $dp[4][7] = 9$ (lấy vật 2 và vật 3: trọng lượng $3 + 4 = 7$, giá trị $4 + 5 = 9$).

Code (2D DP)

C++

```
#include <bits/stdc++.h>
using namespace std;

int knapsack(int n, int W, vector<int>& w, vector<int>& v) {
    vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));
    for (int i = 1; i <= n; i++) {
        for (int j = 0; j <= W; j++) {
            dp[i][j] = dp[i - 1][j]; // không lấy vật i
            if (j >= w[i]) {
                dp[i][j] = max(dp[i][j], dp[i - 1][j - w[i]] + v[i]); // lấy vật i
            }
        }
    }
    return dp[n][W];
}

int main() {
    int n = 4, W = 7;
    vector<int> w = {0, 1, 3, 4, 5}; // 1-indexed
    vector<int> v = {0, 1, 4, 5, 7}; // 1-indexed
    cout << knapsack(n, W, w, v) << "\n";
    return 0;
}
```

Python

```
def knapsack(n, W, w, v):
    dp = [[0] * (W + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for j in range(W + 1):
            dp[i][j] = dp[i - 1][j] # không lấy vật i
            if j >= w[i]:
                dp[i][j] = max(dp[i][j], dp[i - 1][j - w[i]] + v[i]) # lấy vật i
    return dp[n][W]
```



```
n, W = 4, 7
w = [0, 1, 3, 4, 5] # 1-indexed
v = [0, 1, 4, 5, 7] # 1-indexed
print(knapsack(n, W, w, v))
```

Tối ưu bộ nhớ: Bước trung gian — `dp_old` / `dp_new`

Nhận xét: để tính hàng i , ta chỉ cần hàng $i - 1$. Vì vậy, thay vì lưu toàn bộ bảng $(n + 1) \times (W + 1)$, ta có thể dùng 2 mảng 1D:

- `dp_old[j]` — lưu kết quả hàng $i - 1$ (đã tính xong)
- `dp_new[j]` — lưu kết quả hàng i (đang tính)

Công thức lúc này:

$$dp_new[j] = \max(dp_old[j], dp_old[j - w_i] + v_i)$$

Điểm mấu chốt: khi tính `dp_new[j]`, ta chỉ đọc từ `dp_old` — không bao giờ ghi đè nó. Đây là điều sẽ thay đổi ở bước tiếp theo.

C++

```
#include <bits/stdc++.h>
using namespace std;

int knapsack(int n, int W, vector<int>& w, vector<int>& v) {
    vector<int> dp_old(W + 1, 0);
    vector<int> dp_new(W + 1, 0);
    for (int i = 1; i <= n; i++) {
        for (int j = 0; j <= W; j++) {
            dp_new[j] = dp_old[j]; // không lấy vật i
            if (j >= w[i]) {
                dp_new[j] = max(dp_new[j], dp_old[j - w[i]] + v[i]); // lấy vật i
            }
        }
        swap(dp_old, dp_new); // dp_old <- hàng vừa tính
    }
    return dp_old[W]; // sau swap cuối, dp_old chứa kết quả hàng n
}

int main() {
    int n = 4, W = 7;
    vector<int> w = {0, 1, 3, 4, 5}; // 1-indexed
    vector<int> v = {0, 1, 4, 5, 7}; // 1-indexed
    cout << knapsack(n, W, w, v) << "\n";
    return 0;
}
```

Python

```
def knapsack(n, W, w, v):
    dp_old = [0] * (W + 1)
    dp_new = [0] * (W + 1)
    for i in range(1, n + 1):
        for j in range(W + 1):
            dp_new[j] = dp_old[j] # không lấy vật i
            if j >= w[i]:
                dp_new[j] = max(dp_new[j], dp_old[j - w[i]] + v[i]) # lấy vật i
        dp_old, dp_new = dp_new, dp_old # dp_old <- hàng vừa tính
    return dp_old[W] # sau swap cuối, dp_old chứa kết quả hàng n

n, W = 4, 7
w = [0, 1, 3, 4, 5] # 1-indexed
v = [0, 1, 4, 5, 7] # 1-indexed
print(knapsack(n, W, w, v))
```

Bộ nhớ giảm từ $O(nW)$ xuống $O(W)$, và code vẫn trực quan: `dp_old` luôn là "hàng trước", `dp_new` luôn là "hàng đang tính".

Tối ưu bộ nhớ: Gộp thành mảng 1D

Nhìn lại công thức:

$$dp_new[j] = \max(dp_old[j], dp_old[j - w_i] + v_i)$$

Khi tính `dp_new[j]`, ta cần `dp_old[j - w_i]` — một vị trí nhỏ hơn j (vì $w_i > 0$).

Điều này có nghĩa: nếu ta gộp thành 1 mảng `dp[]` và **duyệt j từ lớn đến nhỏ** (từ W về w_i), thì khi cập nhật `dp[j]`:

- `dp[j - w_i]` chưa được cập nhật trong vòng lặp hiện tại (vì $j - w_i < j$ và ta chưa duyệt đến đó)
- $\rightarrow$ nó vẫn là giá trị cũ $\equiv dp_old[j - w_i]$ ✓

Ngược lại, nếu duyệt xuôi (từ w_i đến W), khi tính `dp[j]`, `dp[j - w_i]` đã bị ghi đè bởi vòng lặp hiện tại $\rightarrow$ ta vô tình lấy vật i nhiều lần (Unbounded Knapsack).

C++

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, W;
    cin >> n >> W;
    vector<int> w(n), v(n);
    for (int i = 0; i < n; i++) cin >> w[i] >> v[i];

    vector<int> dp(W + 1, 0);

    for (int i = 0; i < n; i++) {
        for (int j = W; j >= w[i]; j--) { // duyệt ngược
```

```

        dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    }
}

cout << dp[W] << "\n";
}

```

Python

```

n, W = map(int, input().split())
w, v = [], []
for _ in range(n):
    wi, vi = map(int, input().split())
    w.append(wi)
    v.append(vi)

dp = [0] * (W + 1)

for i in range(n):
    for j in range(W, w[i] - 1, -1): # duyệt ngược
        dp[j] = max(dp[j], dp[j - w[i]] + v[i])

print(dp[W])

```

8. Khi nào dùng DP?

Hãy tự hỏi:

1. Bài toán có tính tối ưu không? (tìm min, max, đếm số cách...)
2. Có bài toán con chồng chéo không? (cùng tính toán lặp lại nhiều lần?)
3. Greedy có cho sai không? (nếu greedy đúng, không cần DP)
4. Có thể tìm công thức truy hồi không? ($dp[i]$ tính từ $dp[\dots]$?)

Nếu câu trả lời là "có" → rất có thể đây là bài DP!

9. Sai lầm phổ biến khi làm DP

• Sai lầm 1: Sai cơ sở (base case)

- Quên khởi tạo $dp[0]$, hoặc khởi tạo sai.
- Ví dụ: Coin Change phải có $dp[0] = 0$, nếu quên thì mọi giá trị đều sai.

• Sai lầm 2: Sai thứ tự tính

- Bottom-up phải tính từ nhỏ đến lớn (hoặc theo thứ tự mà các trạng thái phụ thuộc đã được tính xong).
- Knapsack 0/1 phải duyệt ngược j , nếu duyệt xuôi sẽ thành unbounded knapsack.

- **Sai lầm 3: Quên dùng long long**

- Khi đề bài yêu cầu modulo $\rightarrow$ kết quả trung gian có thể vượt `int`.
- Khi đếm số cách mà không modulo $\rightarrow$ kết quả cực lớn.

- **Sai lầm 4: Tràn mảng**

- Truy cập $dp[i - 1]$ khi $i = 0 \rightarrow$ lỗi.
- Truy cập $dp[i - c]$ khi $i < c \rightarrow$ lỗi.

- **Sai lầm 5: Nhầm lẫn "đếm cách" và "tìm min/max"**

- Đếm cách $\rightarrow$ khởi tạo $dp[0] = 1$, cộng dồn.
- Tìm min $\rightarrow$ khởi tạo INF, dùng min.
- Tìm max $\rightarrow$ khởi tạo 0 hoặc -INF, dùng max.

10. Mẹo luyện tập

- Bắt đầu từ bài đơn giản: Fibonacci $\rightarrow$ Coin Change $\rightarrow$ Grid Paths $\rightarrow$ LIS $\rightarrow$ Knapsack.
- Luôn viết truy hồi trên giấy trước khi code.
- Tự chạy tay bảng DP với ví dụ nhỏ để kiểm tra.
- Nếu stuck: thử suy nghĩ "nếu đồng xu cuối cùng / vật cuối cùng / phần tử cuối cùng là gì?".
- Đừng nản! DP cần thời gian để quen. Giải 20-30 bài DP, bạn sẽ bắt đầu nhìn ra pattern.